Борувка на SPLA и Apache Spark. Результаты

Шальнев Владислав, Кадочников Даниил

Характеристики машины Эксперимент 1

ОС: Ubuntu 24.04.4 LTS

CPU: Intel Core i5-10300H @ 2.50 ГГц

- 4 ядра, 8 потоков (Hyper-Threading)
- L1: 256 КБ, L2: 1 МБ, L3: 8 МБ

GPU:

- NVIDIA GeForce GTX 1650 Ti Mobile (4 ГБ GDDR6);
- Intel UHD Graphics (Comet Lake-H GT2), встроенная

RAM: 16 ГБ DDR4, 3200 МГц

Программное обеспечение (Эксперимент 1)

- NVIDIA драйвер 580.126.09
- CUDA Toolkit 13.0
- SPLA (commit b81d1e2)
- Apache Spark 3.5.1
- gcc / g++ 13.3.0
- OpenJDK 11.0.30
- OpenCL 3.0
- пакет `intel-opencl-icd` 23.43.27642.40-1ubuntu3 (Neo)
- пакет `pocl-opencl-icd` 5.0-2.1build3
- CMake 3.28.3

Данные Эксперимент 1

Граф	Количество вершин	Количество рёбер
amazon0601	403 394	2 443 408
citationCiteseer	268 495	1 156 647
coPapersDBLP	540 486	15 245 729
in-2004	1 382 908	13 591 473
cit-Patents	3 774 768	16 518 947

Источники: A High-Performance MST Implementation for GPUs 2023, SuiteSparse Matrix Collection

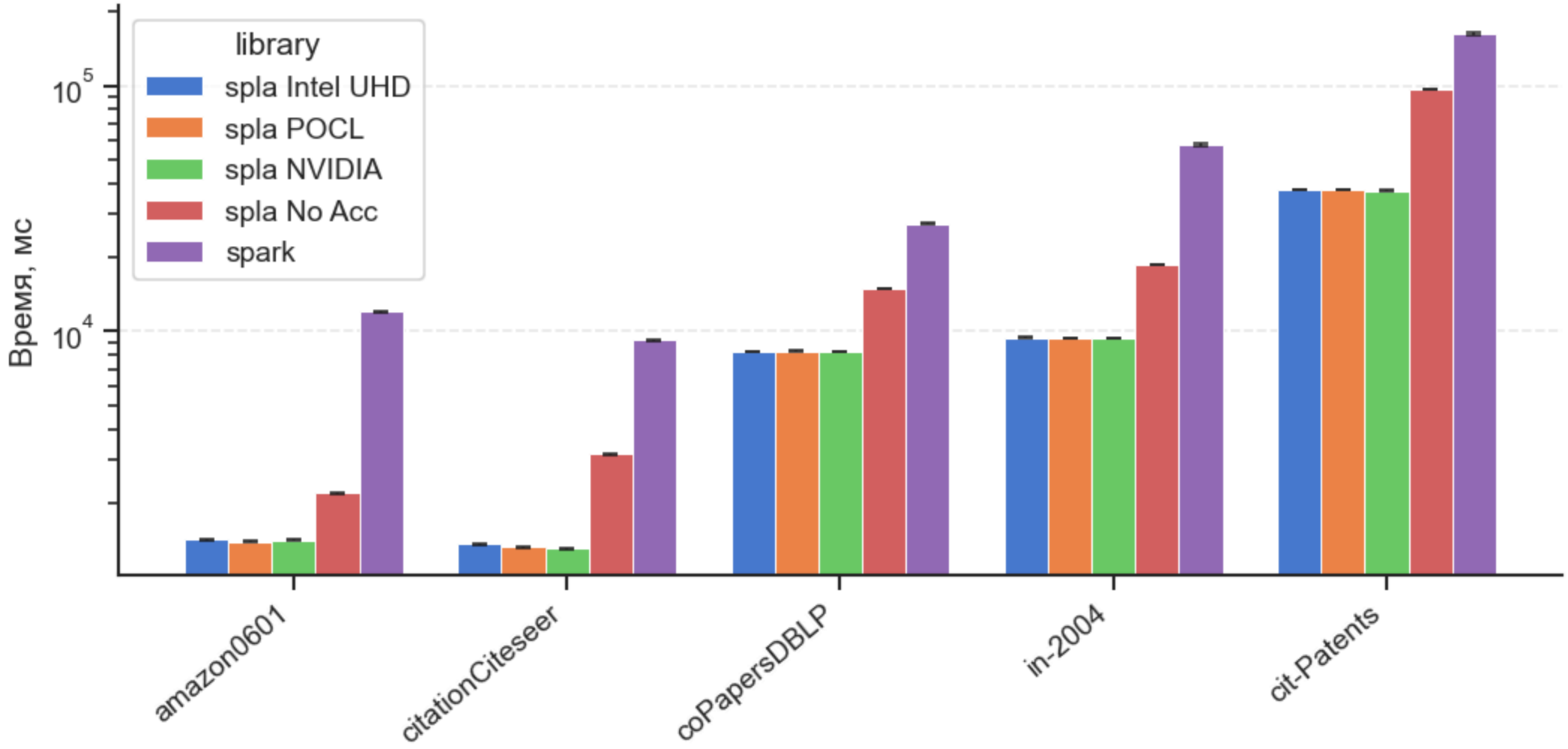
Эксперимент 1

Цель: сравнить время работы алгоритма Борувки на SPLA (No Acc, Intel UHD, POCL, NVIDIA) и Apache Spark на реальных графах разного размера

Ход экспериментов

- 15 запусков алгоритма на каждом графе и библиотеке, 5 прогревов
- Реализация на Spark запускается с `--cores=8`
- Вычисление среднего времени и доверительных интервалов
- Визуализация и анализ результатов

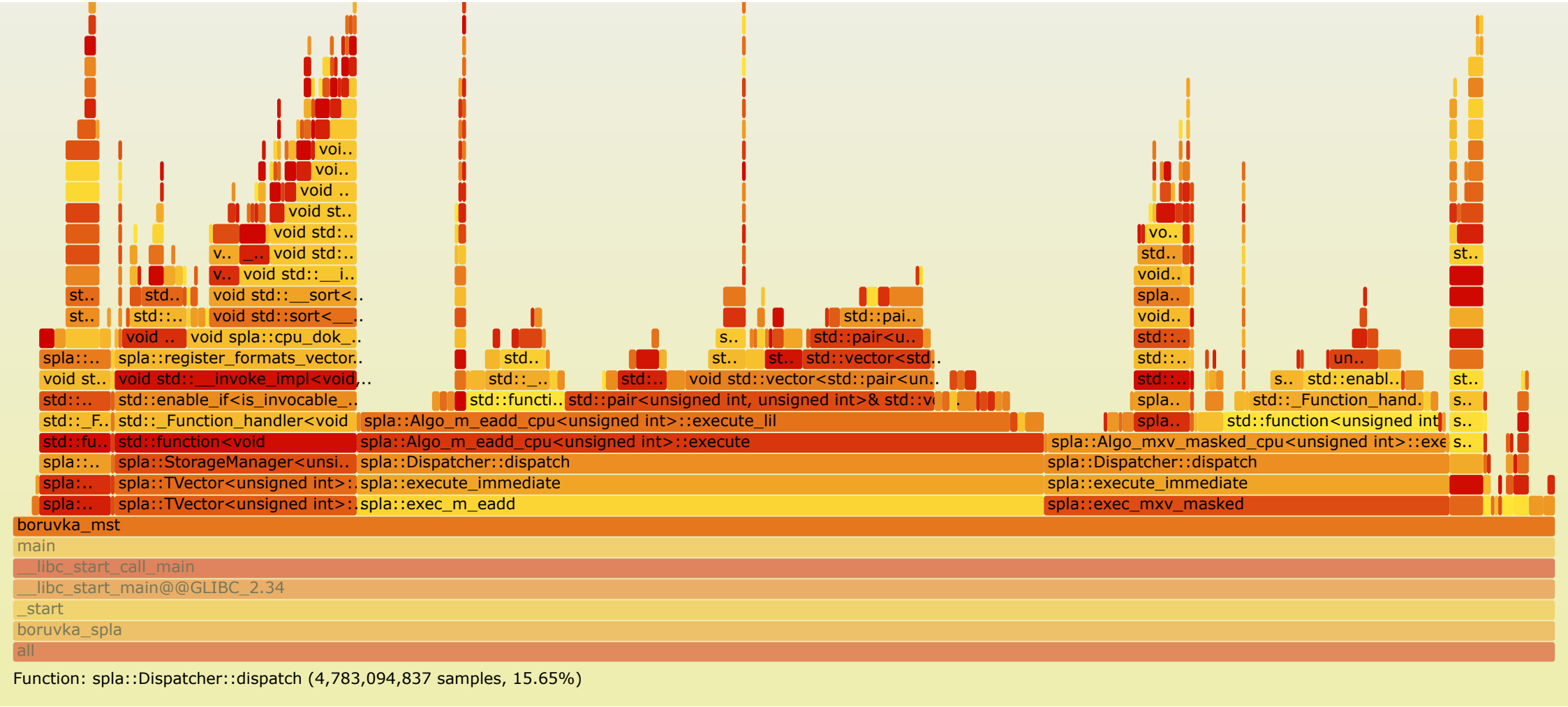
Эксперимент 1



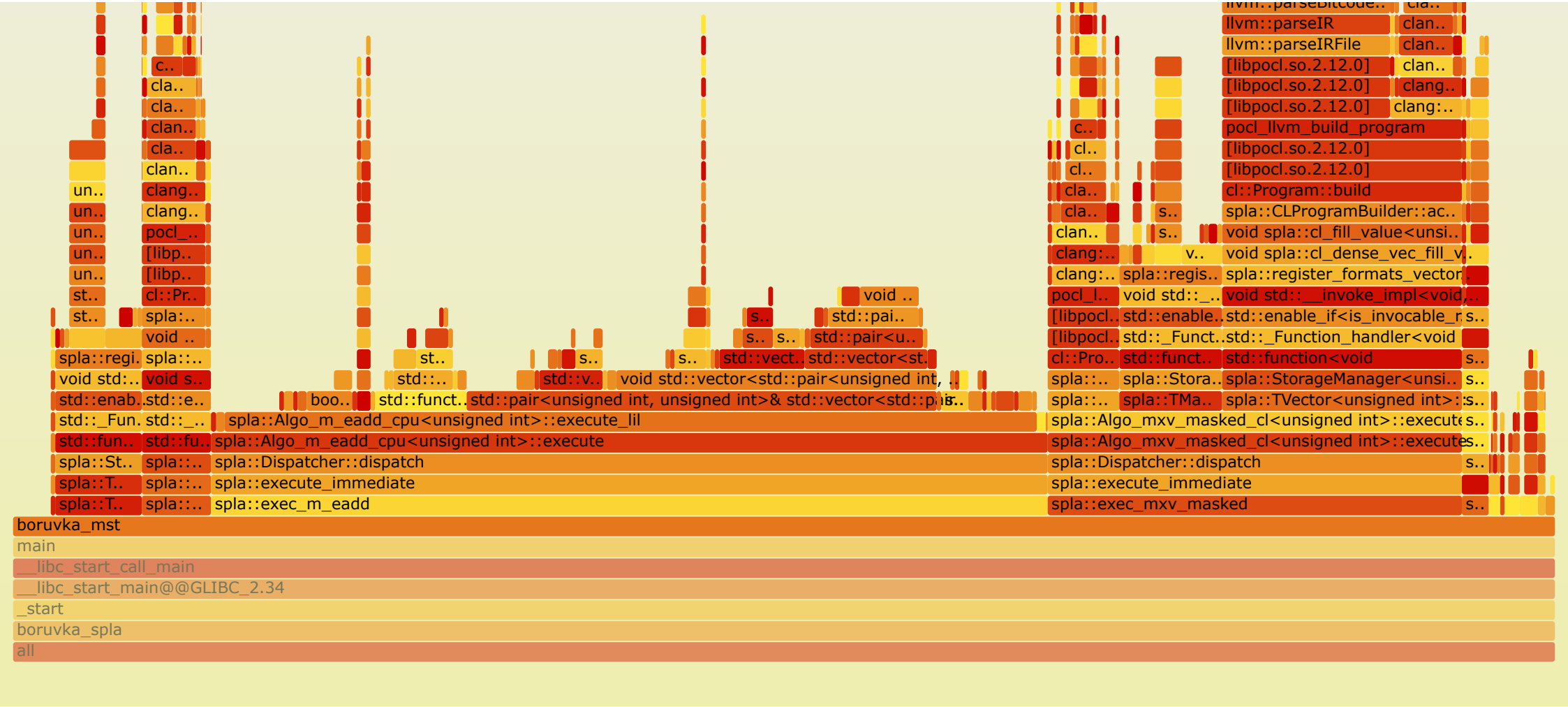
Результаты

- **Intel UHD, NVIDIA и POCL:** время на всех пяти графах **близкое** (расхождение до **~4%**).
- **Относительно SPLA CPU** все три OpenCL-бэкенда **быстрее** примерно **в 1,6–2,6 раза** (минимум на amazon0601, максимум на cit-Patents).
- **Относительно Spark** лучший SPLA (OpenCL) **быстрее** примерно **в 3–9 раз**; наибольший запас на **amazon0601** (**~8,6×**), наименьший на **coPapersDBLP** (**~3,1×**).

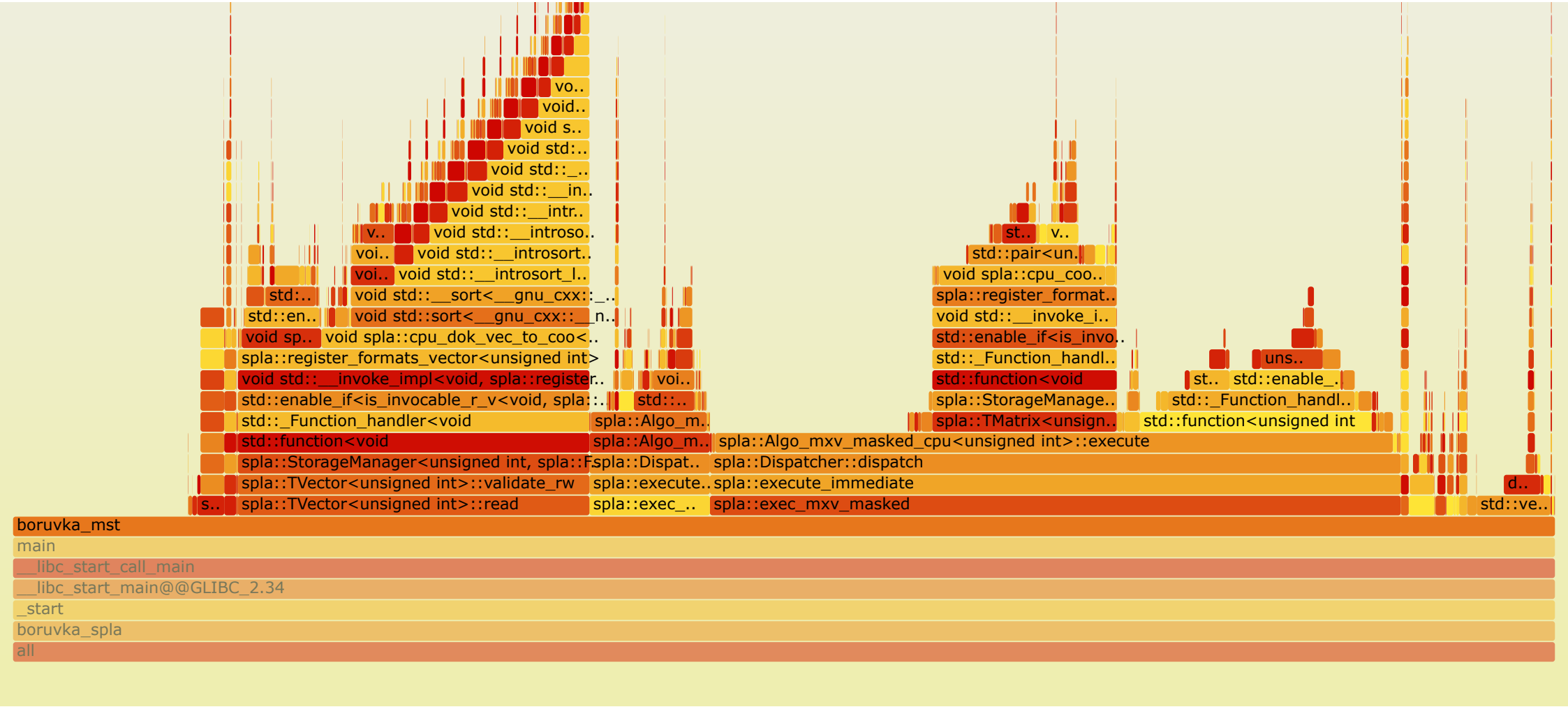
Профилирование SPLA (amazon0601.mtx, No Acc)



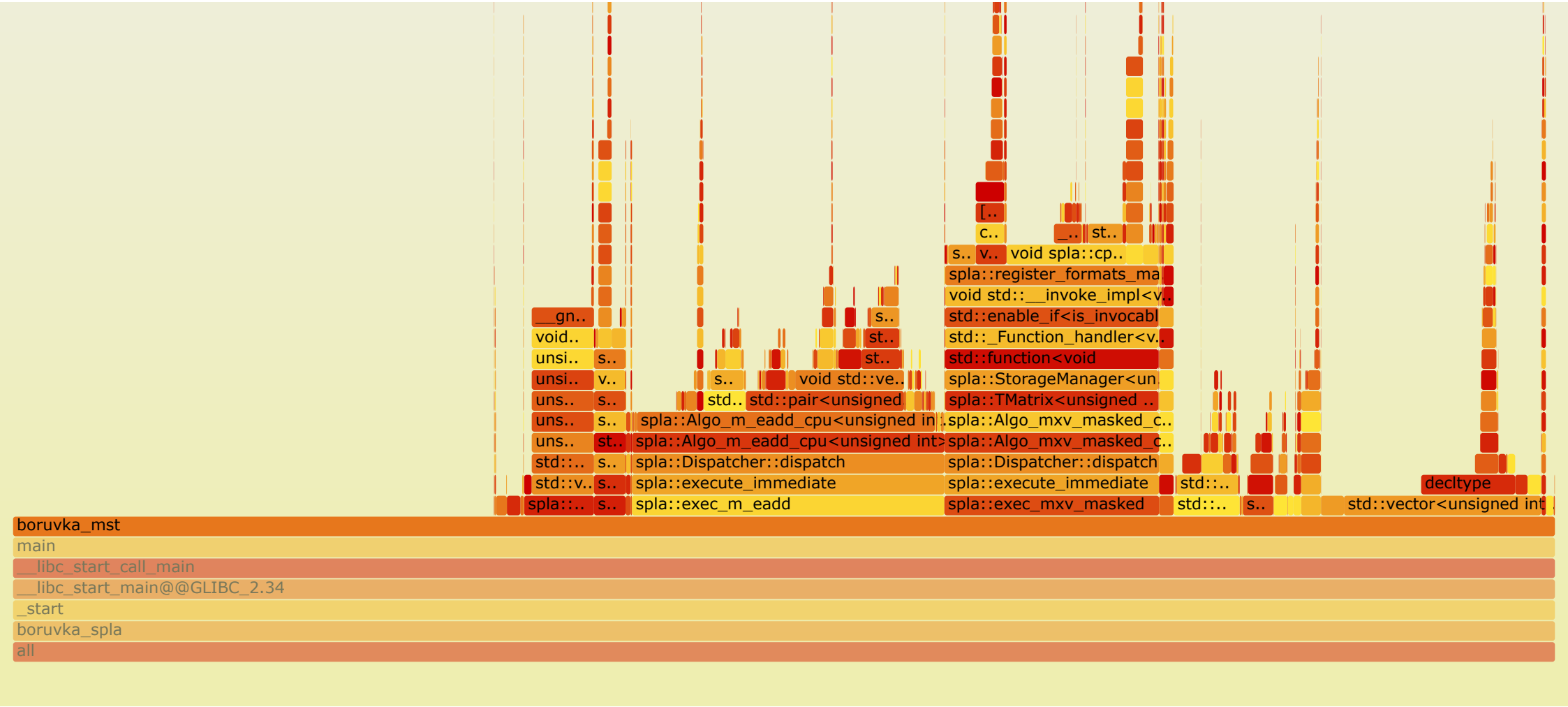
Профилирование SPLA (amazon0601.mtx, POCL)



Профилирование SPLA (cit-Patents.mtx, No Acc)

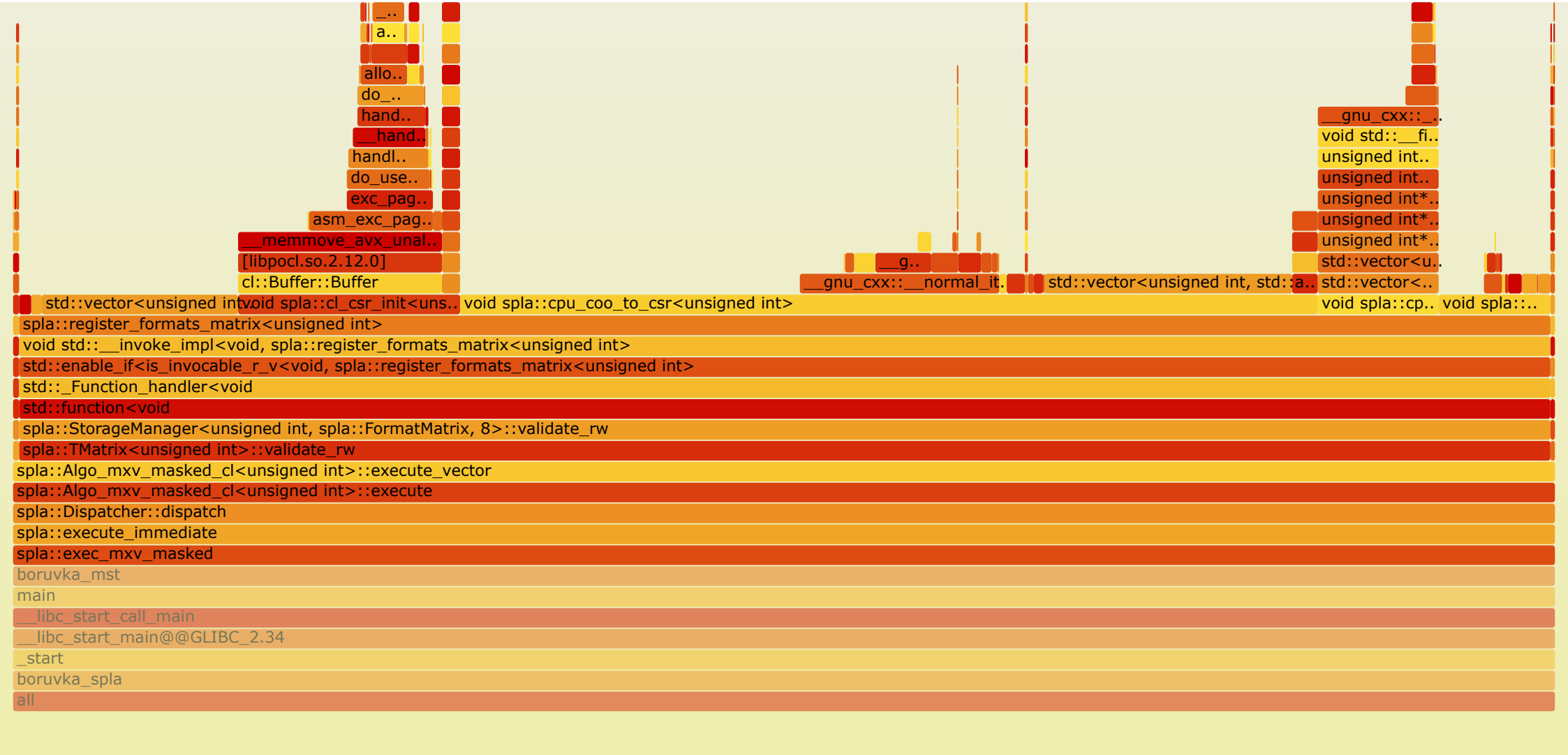


Профилирование SPLA (cit-Patents.mtx, POCL)



SPLA (amazon0601.mtx, POCL, mxv)

SPLA (cit-Patents.mtx, POCL, mxv)



Анализ результатов

- SpMV (`exec_mxv_masked`) — по профилю один из самых затратных шагов; **на OpenCL даёт значительное ускорение** относительно SPLA CPU
- **Ускорение от OpenCL растёт с размером графа** (amazon0601: ~1.5×, cit-Patents: ~2.6×): фиксированные накладные расходы на сборку, запуск кернела и синхронизацию незначительны при большой матрице S
- `exec_m_eadd` (в т.ч. построение рабочей S из A) — **существенная доля** времени; **OpenCL-реализации нет**, выполнение на CPU
- **Чего не хватало в SPLA**: **пользовательских типов** и полукольца на кортежах (w, idx) без упаковки в `uint32`; примитивов **scatter** (аналог `FastAssign`) и **gather** (`t[i]=f[f[i]]`) `select` для фильтрации матрицы

Характеристики машины Эксперимент 2

ОС: Windows 10 Pro (22H2)

CPU: 12th Gen Intel Core i5-1235U @ 1.30 GHz

- Ядра: 10 (2 производительных + 8 эффективных)
- Потоки: 12
- L3 Кэш: 12 МБ

RAM: 32 ГБ DDR4, 3200 МГц

OpenCL:

- Platform: Intel(R) OpenCL
- Device: 12th Gen Intel(R) Core(TM) i5-1235U
- Driver Version: 2025.20.10.0.23_160000

Данные Эксперимент 2

Данные эксперимента: Road Networks

Граф	Вершины	Рёбра
road-usroads	129 164	165 435
USA-road-d.NY	264 346	730 100
USA-road-d.COL	435 666	1 042 400
road-belgium-osm	1 038 823	1 549 970
road-roadNet-CA	1 957 027	2 760 388

Данные эксперимента: Geometric Graphs

Граф	Вершины	Рёбра
delaunay_n17	131 072	393 176
delaunay_n18	262 144	786 396
delaunay_n19	524 288	1 572 823
delaunay_n20	1 048 576	3 145 686
delaunay_n21	2 097 152	6 291 408

Данные эксперимента: Social & Collaboration Networks

Граф	Вершины	Рёбра
soc-sign-epinions	131 828	840 799
ca-citeseer	227 320	814 134
soc-flickr	513 969	3 190 452
soc-lastfm	1 191 805	4 519 330
soc-flickr-und	2 394 385	15 555 042

Источник: Network Repository (NRVIS)

Эксперимент 2

Цель: исследовать влияние типа графа на относительную производительность реализаций алгоритма Борувки на SPLA (CPU) и Apache Spark

Вопрос: зависит ли соотношение времени работы SPLA и Spark от структурных свойств графа?

Типы графов:

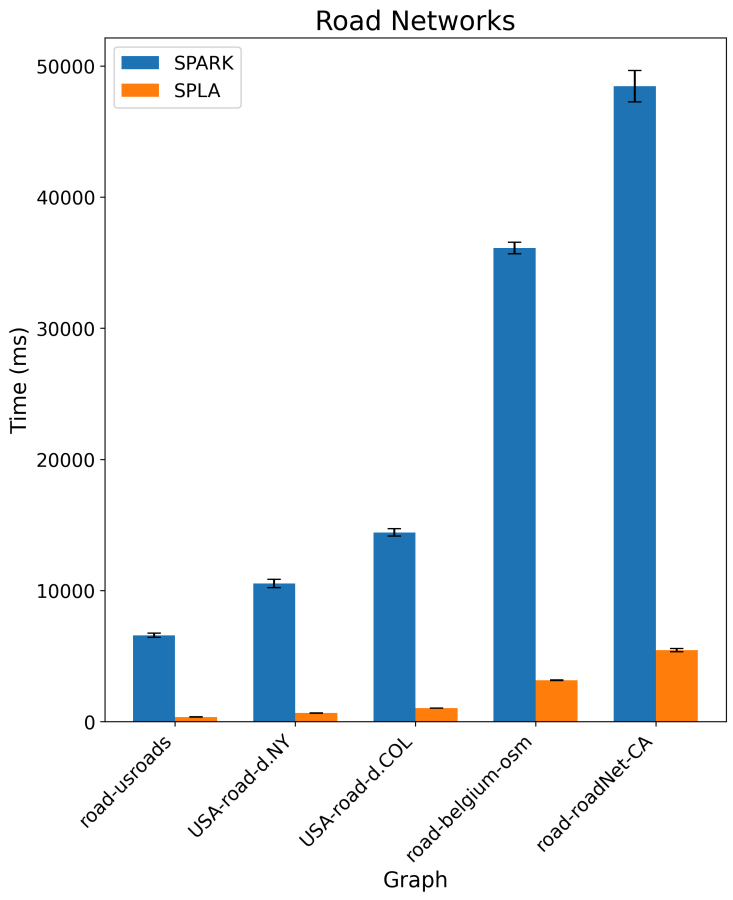
- **Road** — разреженные дорожные сети, много итераций Борувки
- **Geometric** — триангуляции Делоне, средняя плотность
- **Social** — социальные/коллаборационные сети, высокая плотность, мало итераций

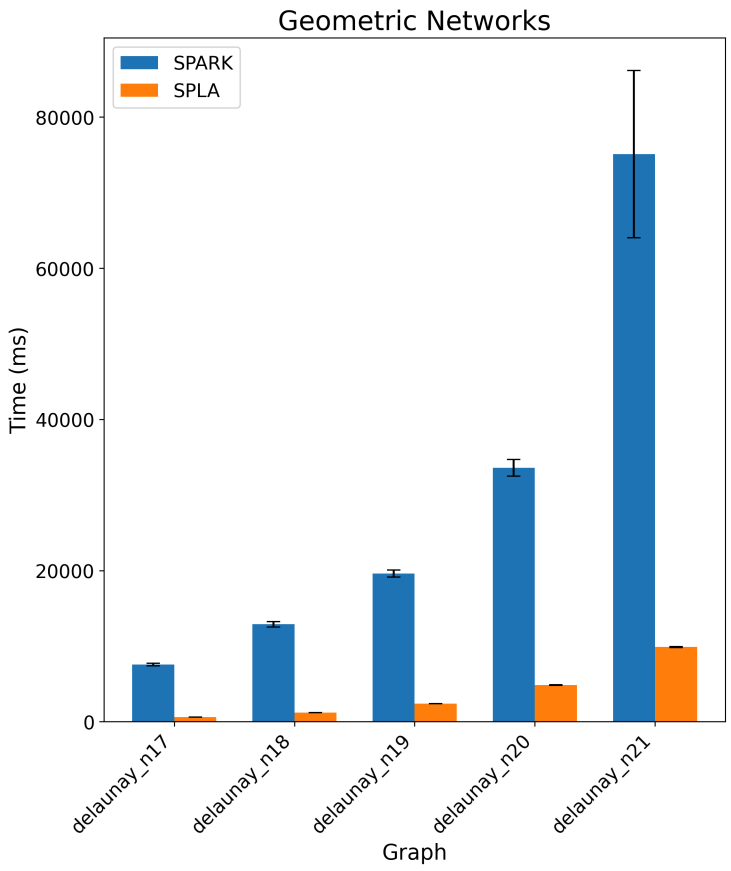
Ход эксперимента 2

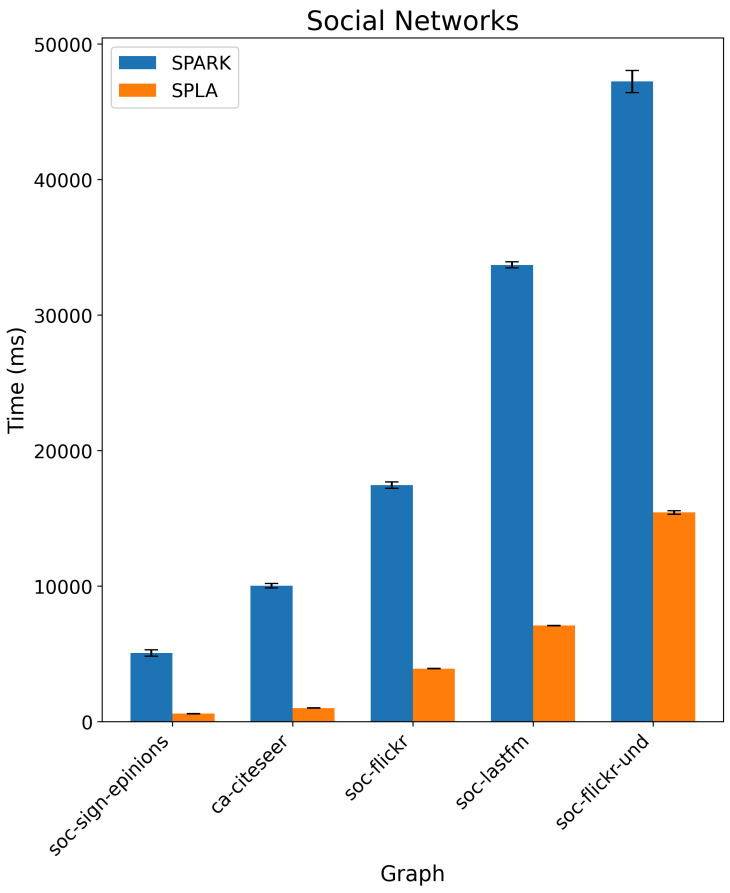
Влияние типа графа

- 5 запусков алгоритма на каждом графе и библиотеке, 3 прогрева
- Сравнение SPLA (CPU) и Spark
- Визуализация и анализ результатов

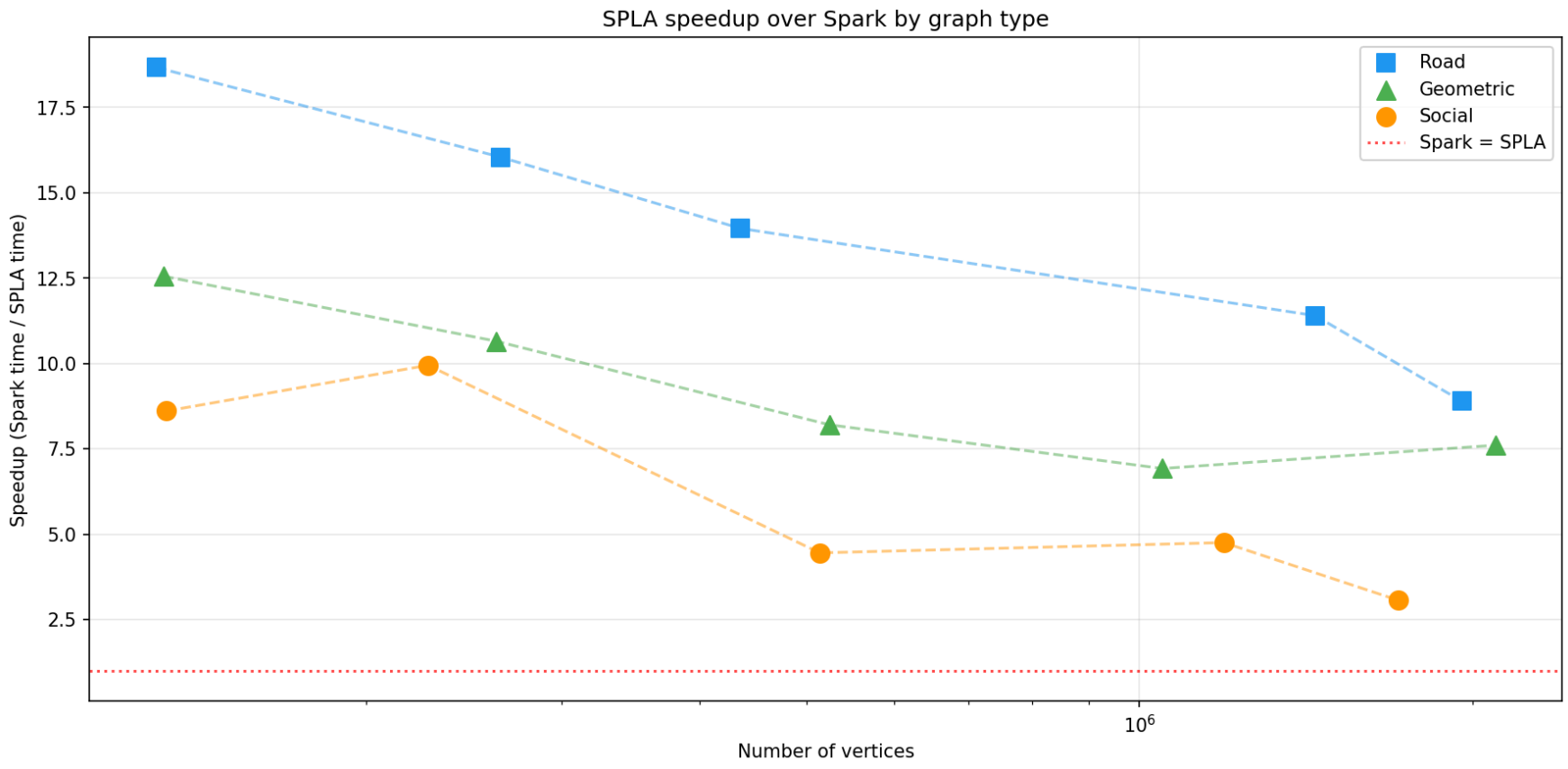
Эксперимент 2 — Сравнение по типу графа



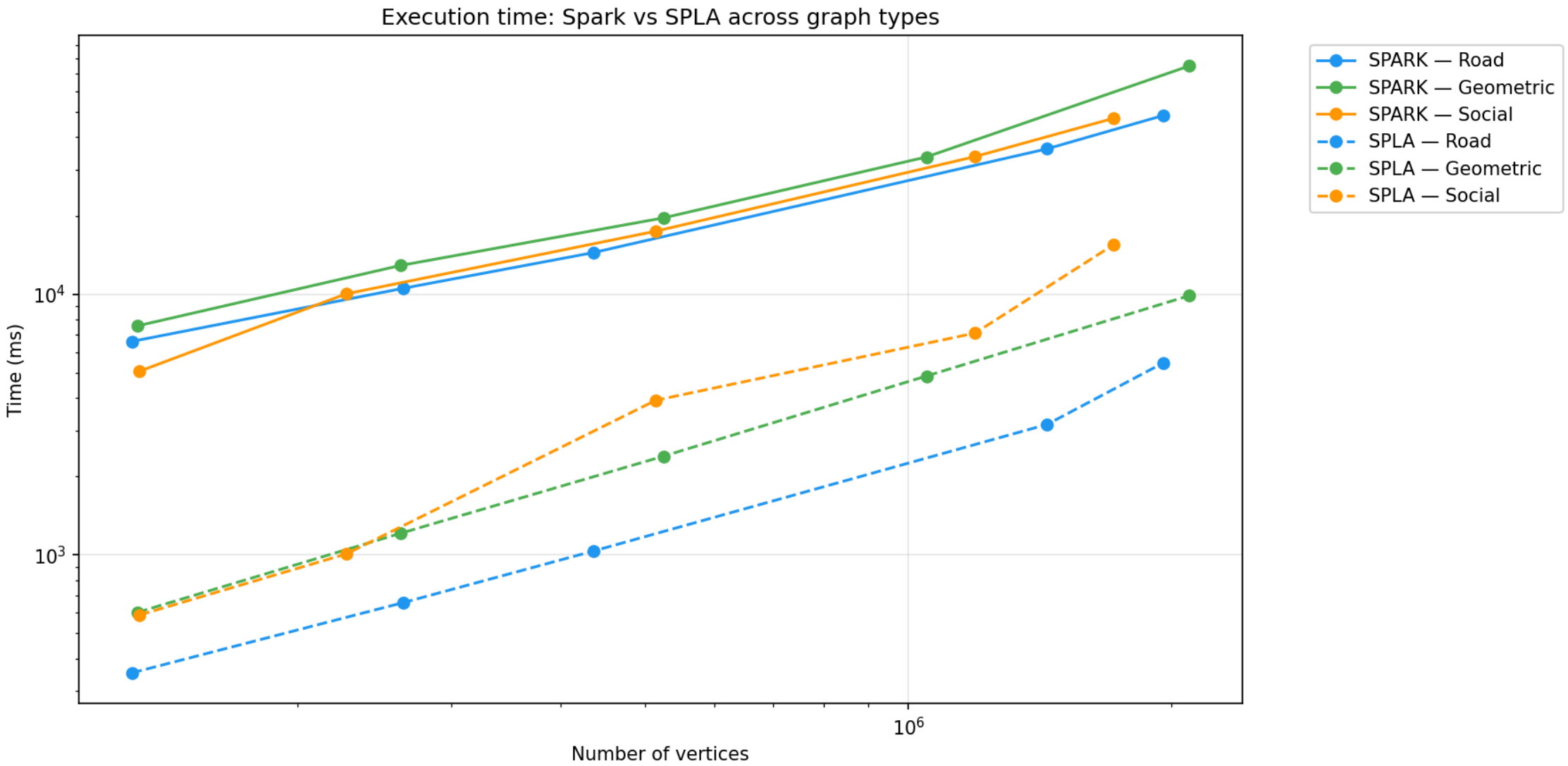




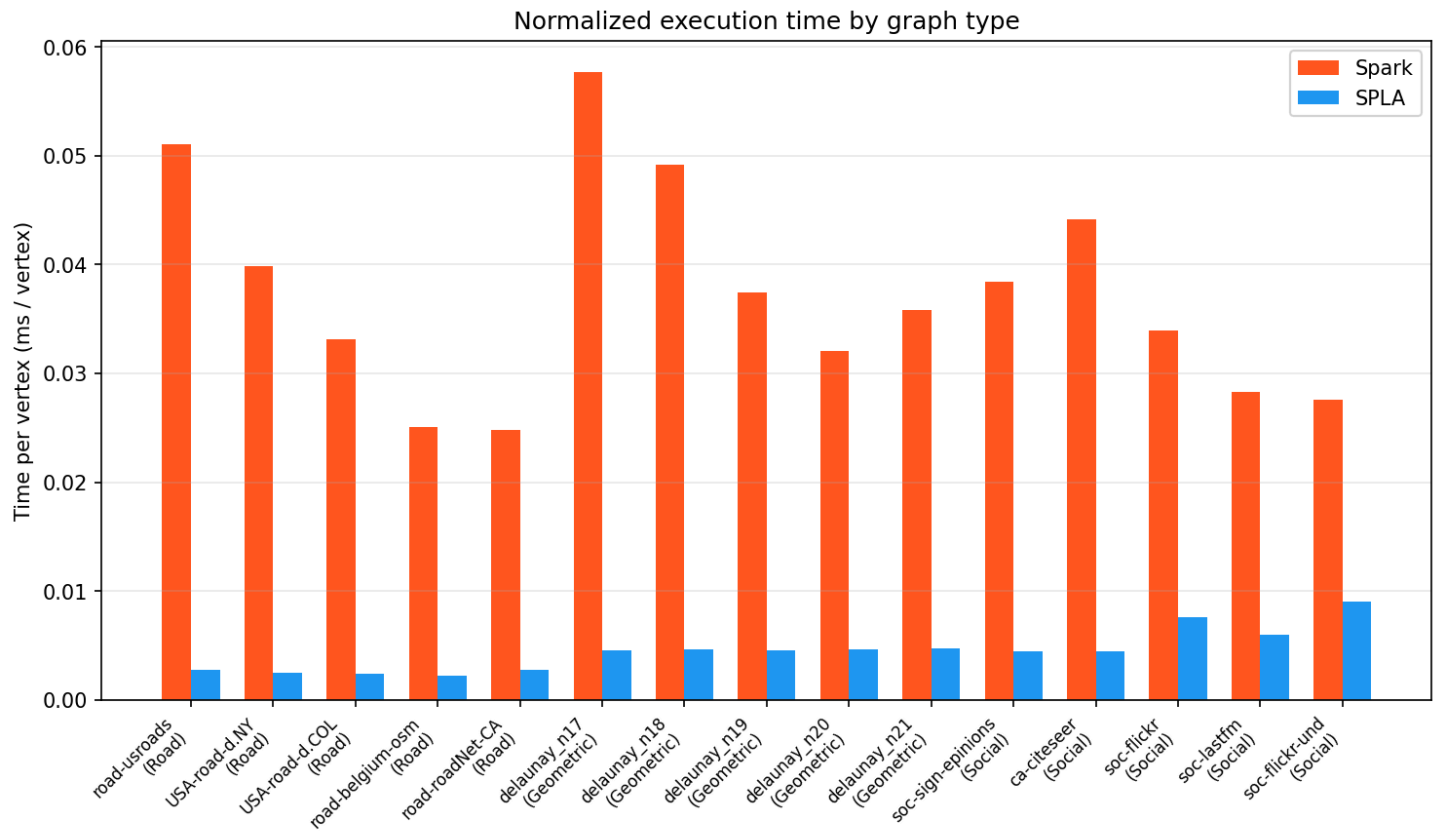
Эксперимент 2 — Ускорение SPLA относительно Spark



Эксперимент 2 — Время по размеру и типу графа



Эксперимент 2 — Нормализованное время (ms/вершину)



Выводы Эксперимент 2

1. **Тип графа влияет на соотношение производительности** SPLA и Spark
2. **Road-графы** — наибольший разрыв: Spark тратит больше на многочисленные итерации
3. **Social-графы** — наименьший разрыв: мало итераций, overhead Spark менее заметен
4. **Главный фактор** — число итераций алгоритма Борувки, которое определяется структурой графа
5. При выборе реализации MST следует учитывать **тип входных данных**